

Evaluating Non-Generative Decision Models in Sequential Gridworld Tasks

Mateus Pereira
Independent researcher

Abstract

Some AI models do not write text: given a situation and a list of possible actions, they return a probability for each action. Can such non-generative decision models guide an agent all the way to a goal? System-One Control Bench tests this on 100 grid puzzles in which a piece must reach a goal, sometimes by way of a key and a locked door. An exact solver knows every best move. We vary what the model is told (ten input conditions) and how many steps one move contains (five move rules), and compare three decision models (Jev, Laya and GLiClass) with four chat-model setups and simple baselines. With full context (every extra description), Jev reaches the goal in 57% of games, close to Gemma 4 26B (61%) and DeepSeek V4.1 Flash (60%); DeepSeek V4.1 Flash with reasoning does best, at 80%. Telling Jev what each move would do raises its success rate by 16 percentage points over the map only. Moves of several steps help or hurt depending on the model, the input and the rule. On an exam of 495 fixed positions, Jev with full context picks an optimal move 90% of the time, yet it wins only 57% of its games: good single decisions do not guarantee reliable navigation.

1 Introduction

AI models are increasingly asked to act step by step: choose an action, see what happens, choose the next. A model can choose well when shown a single situation and still fail when its own choices decide what it sees next. We test this in a small grid world where the best moves are known exactly. We focus on **non-generative decision models**. Such a model does not write an answer. It receives a description of the situation, a question and a list of options, and returns a probability for each option; the program using it plays the most likely one. Jev, from TypeSafe, is one such model. It has been tried on choosing chess moves [1] and on controlling phone apps and web browsers [2], [3]. We also test Laya’s Typed-Decisions model, an open model built to answer the same kind of request, and GLiClass, an open classifier that can score any list of labels. We compare them with chat models, which normally write their answers, and with simple baselines. The benchmark is called System-One Control Bench after “System 1”, the fast kind of thinking: these models choose an action directly, without writing out any reasoning first.

In every puzzle the map holds all the information needed to reach the goal. We change two things: what extra descriptions the model gets, and how many steps one move contains. Five questions guide the report:

1. **RQ1:** How often do models reach the goal?
2. **RQ2:** Which extra descriptions help?
3. **RQ3:** Do good answers on fixed positions translate into won games?
4. **RQ4:** What changes when one move contains several steps?
5. **RQ5:** What do the returned probabilities tell us, and what do the models cost?

All puzzles, the solver, every request and every game record are public, so every result can be checked and recomputed.

2 Related work

Jev has been tested as a chess-move picker [1] and compared with Laya and a local Qwen3 decision model [4]. The Laya model we test was trained on four kinds of workflow task, and its model card warns that it may not transfer to others [5]. GLiClass classifies a text against any list of labels in a single pass [6]; using the option texts as labels turns it into a player.

PlanBench tests planning in formal domains, where answers can be checked exactly [7]. AgentBench [8] and BALROG [9] evaluate agents across many environments and games. We instead use one small environment and vary what the model is told, to find out what these models need. BabyAI [10] and MiniGrid [11] are related grid worlds. Their agents turn and move forward; our moves (one step north, south, east or west) need no sense of facing, so the task is about reading the map. SayCan combines a language model’s choice of skill with an estimate of whether the skill can succeed [12], and SayCanPay adds learned knowledge of the domain [13]. Our move outcomes (option texts that say what each move would do) are simpler: the environment’s own code writes what each move would do. Multiple-choice answers are known to depend on the order and labels of the options [14], which is why we shuffle them. Calibration measures whether stated confidence matches how often a model is right [15].

3 Benchmark design

3.1 Puzzles

Each puzzle is a small grid map (Figure 1). A piece, **a**, must reach the goal, **g**, and walls, **#**, are in the way. Some puzzles also have a key, **κ**, and a locked door, **Ⓛ**: the piece picks up the key by stepping onto it, and the door opens only when the piece walks into it carrying the key. A **step** moves the piece one cell north, south, east or west. A step into a wall, or into the door without the key, is **blocked**, and the piece stays where it is.

An exact solver (a breadth-first search over the piece’s position, the key and the door) knows how many steps each position is from the goal. We call a move **optimal** if it starts a shortest route to the goal. When several moves are equally good, all of them count as optimal.

The 100 puzzles are grouped into levels 1, 2, 3, 4, 5, 6, 8, 10, 12, 15 and 20, where the level is the length of the shortest route in steps. Levels 1 and 2 have five puzzles each, and the others ten. A seeded generator made rooms and mazes; one level-10 maze is hand-made. Keys appear from level 3, and every puzzle from level 10 up has one. Higher levels also need detours away from the goal. Greedy (walls), a baseline that always steps toward its next target, loses every puzzle from level 10 up. Half of the puzzles at levels 12, 15 and 20 have extra rings of outer wall, which make the map text longer without changing the puzzle.

0123456789 0 ##### 1 #.AX# 2 #. . ##**K.# 3 #D****#. . # 4 #G#. #. . . ## 5 #####	Map only: the map, the position (7, 1), “You are carrying nothing”, where the objects are, and “What is the best next move?” With move outcomes: each option says what it does, such as “move south (down): you move to (7, 2) and pick up the key”. Shortest route: south, west, west, south, west, west, west, west, south (nine steps).
--	---

Figure 1: An example position. Stars trace a shortest route through K and D to G; X marks where moving east leads, which is not optimal. Stars and X appear only in this figure. Appendix A shows two complete requests for this position.

3.2 Move rules

A **move** is one decision by the model. Under the basic **compass** rules, a move is a single step. Under the other four, the **sequence rules**, a move can be a sequence of two or three steps (Table 1), and every possible sequence is offered. Two of them fix the number of steps (**two-moves**, **three-moves**); the two **up-to** rules also allow shorter moves. The steps run one after another: a blocked step is wasted and the rest still run, and reaching the goal ends the move at once. The model sees the new position only after the whole move.

Longer moves change several things at once: the number of options, the length of each option’s text, and how many steps happen before the model sees the board again. Several sequences can also end in the same place. We compare the rules as a whole and do not separate these effects.

A game is won when the piece reaches the goal. It is lost when the model has used twice as many moves as the solver needs under the same rules.

Table 1: The five move rules. Every sequence of steps is offered, including ones that walk into walls.

Rules	One move is	Options
compass	one step	4
two-moves	exactly two steps	16
up-to-two-moves	one or two steps	20
three-moves	exactly three steps	64
up-to-three-moves	one, two or three steps	84

3.3 What the model is told

Every request contains the rules of the game, the map with numbered rows and columns, the piece’s position, whether it carries the key, and where the key, door and goal are. We call this basic request **map only**. Four **components** can be added to it:

- **Surroundings:** what is next to the piece in each direction, and how far away each object is (“The key is 1 south of you”).
- **Move history:** every move so far and what it did.
- **Move outcomes:** each option also says what it would do (“move south (down): you move to (7, 2) and pick up the key”).
- **Subgoal:** instead of “What is the best next move?”, the question names the next target: the key, then the door, then the goal.

Surroundings, move outcomes and the subgoal are worked out from the map, so they add no new information; they only make it easier to use. The move history is not needed to know where the piece is, but it can show that the piece is going in circles.

There are ten input conditions (Table 2): map only, the map plus one component, all four components (**full context**), and full context minus one component. Most results compare map only with full context. For short, we call the input condition the **input**. Appendix A shows two complete requests.

Table 2: The ten input conditions. Full context has all four components.

Condition name in the code	Name in this report
map	map only
map+surroundings	map + surroundings
map+memory	map + move history
map+lookahead	map + move outcomes
map+subgoal	map + subgoal
everything	full context
everything-surroundings	full context minus surroundings
everything-memory	full context minus move history
everything-lookahead	full context minus move outcomes
everything-subgoal	full context minus subgoal

3.4 The exam

In a game, a model’s own choices decide which positions it sees next, so after the first move two models no longer answer the same questions. The exam removes this difference. It has 495 fixed positions, five from each puzzle (four at level 1), that every player answers. Every model answers each position once with map only and once with full context (the baselines, which ignore the input, answer once), under compass rules, and we check whether the answer is optimal. The exam score is the share of optimal answers, with each puzzle counting equally. Each position is reached by playing a fixed list of moves from the start through the normal game, so a full-context request also lists those moves in its history.

There are five kinds of position: the **start**; **on-route**, partway along a shortest route; **off-route**, one wrong move away from a shortest route; **after-blocked**, just after a move into a wall; and **late**, in the second half of the route and after the key where there is one. Each puzzle gets one position of each kind where its layout allows; the remaining places are filled with extra off-route or after-blocked positions (Appendix C gives the counts). A mistake on the exam costs nothing later, so the exam measures single decisions, not whole games.

4 Players and measures

4.1 Players

We call anything that plays a game a **player**: seven model setups and four baselines (Table 3). The models see only the text of each request. The baselines read the board directly and ignore the input condition.

Table 3: The players.

Player	What it is and how it chooses
<i>Non-generative decision models: return a probability for each option; we play the highest</i>	
Jev	TypeSafe’s hosted decision model (jev-1.13.0).
Laya	Laya’s Typed-Decisions model, an open model built to answer the same kind of request as Jev, run locally [5].
GLiClass	An open classifier that can score any list of labels without training on them [6], run locally. It scores each option text as a label; we scale the scores to sum to 1.
<i>Chat models</i>	
Qwen3.5-4B	A small open chat model [16], run locally. It writes nothing: we read the probability it gives each option number and play the highest.
Gemma 4 26B	Answers with an option number [17].
DeepSeek V4.1 Flash	Answers with an option number, without reasoning [18].
DeepSeek V4.1 Flash (reasoning)	The same model, allowed to reason for up to 1,024 tokens before answering.
<i>Baselines</i>	
Solver	Always plays an optimal move.
Greedy	Steps toward its next target (the key, then the door, then the goal), ignoring walls.
Greedy (walls)	Like Greedy, but never walks into a wall.
Random	Picks one of the offered options at random.

Random shows what chance achieves: it wins 4 of 100 compass games, all at levels 1 to 3. It is the reference for a player that does not use the input at all. Greedy and Greedy (walls) show how far one gets by heading straight for the target without planning; they play compass moves only. Random and Solver play under every move rule.

Every model gets the same request: the state, the question and the options. Jev receives these as the fields of its API. Laya receives the state, the question and the option texts without their numbers, because it would otherwise spend part of each option’s allowance of 48 tokens (the word pieces a model reads) on the number. Gemma 4 26B and both DeepSeek V4.1 Flash setups [19] run through OpenRouter, always on the DeepInfra host. They receive the request as one chat message, and their answer must be JSON naming one option. Qwen3.5-4B receives the same chat, followed by the start of that JSON answer, {"option": "option_". Qwen3.5-4B, Laya and GLiClass gave identical answers in our replay checks; Jev and DeepSeek V4.1 Flash did not (Appendix C); Gemma 4 26B and DeepSeek V4.1 Flash (reasoning), which also sample with the provider’s default settings, were not checked.

The options are shuffled at every move, with a seed made from the puzzle and the move number, because models can prefer an option for its position or its label [14]. If a model gives no valid answer, the game ends and counts as a loss. Some calls failed because of temporary failures such as network errors; we replayed those games, so the final records have no errors in any of the 282 combinations of player, move rules and input condition. Every model played compass rules under all ten conditions. Jev, Laya, GLiClass, Gemma 4 26B and Qwen3.5-4B also played the four sequence rules. Both DeepSeek setups played compass only, to limit cost.

Before the main runs, we wrote down seven hypotheses about Jev. Appendix B lists them with the results.

4.2 Measures

Every condition has 100 games per player, one per puzzle.

- **Success rate:** the share of games won.
- **Progress:** how close to the goal the piece ever got. Distance is the number of steps the solver would still need from a position, and progress is $1 - (\text{closest distance reached} / \text{distance at the start})$. It is 0 if the piece never got closer than where it started, and 1 if it reached the

goal. Because it uses the closest point, even aimless wandering earns some progress: Random’s is 0.25.

- **Final-state progress:** the same, but measured where the game ended. It is negative if the piece ended farther from the goal than it started.
- **SPL** (success weighted by path length) [20]: 0 for a lost game; for a won game, the fewest moves needed divided by the moves used. An SPL of 1 means every game was won by a shortest route.
- **Blocked moves** (compass rules only): the share of moves that change nothing, because the step runs into a wall or into the locked door without the key.
- **Optimal-move rate:** the share of a player’s moves that are optimal, worked out for each game and then averaged over games.

4.3 Statistics

Intervals. The 95% intervals in brackets show how much a result depends on which puzzles happen to be in the set. We compute them by resampling: draw 100 puzzles from the 100 at random, allowing repeats, recompute the result, repeat this 10,000 times, and keep the middle 95% of the results (a bootstrap interval). These intervals do not show how much a rerun would vary.

Comparisons. Two inputs or two models are always compared on the same 100 puzzles, puzzle by puzzle, and the difference gets an interval computed the same way. For the effect of each component on success rate, we also use McNemar’s exact test. It looks only at the puzzles won with one input and lost with the other, and asks how likely such a lopsided split would be by chance. Each model gets eight of these tests, so we apply the Holm correction, which raises the p-values so that the chance of any false alarm among the eight stays at 5%. We call an effect **significant** when its corrected p-value is below 0.05.

Probabilities. Jev, Laya, GLiClass and Qwen3.5-4B give a probability for every option. We check whether the probability of the chosen move matches how often such moves are optimal (calibration [15]). For the **expected calibration error** (ECE), the moves are sorted into ten bins by that probability (0–0.1, 0.1–0.2, and so on). In each bin we compare the average probability with the share of optimal moves, and ECE is the average gap, weighted by the size of each bin. The **Brier score** is the average squared difference between the probability and the outcome (1 if the move was optimal, 0 if not). For both, lower is better and 0 is perfect. When two moves are equally good, a model that gives each a probability of 0.5 looks underconfident on these measures, although either choice is optimal.

5 Results

5.1 Games won

DeepSeek V4.1 Flash (reasoning) wins the most games: 80% [72, 87] with full context and 67% [57, 76] with the map only (Table 4, Figure 2). With the same input, its intervals do not overlap those of any other model.

With full context, Jev, Gemma 4 26B and DeepSeek V4.1 Flash win 57%, 61% and 60% of games. Compared puzzle by puzzle, Jev wins 4 percentage points fewer than Gemma 4 26B [−10, 2] and 3 fewer than DeepSeek V4.1 Flash [−11, 5]. Both intervals include zero, so neither comparison shows a difference in success rate between Jev and the chat model. With the map only, Jev wins 32%, between Random (4%) and Greedy (walls) (37%). Laya and GLiClass win 2% and 3% with the map only, no better than Random, and 12% and 13% with full context. With the map only, their progress (0.10) is even below Random’s (0.25). Qwen3.5-4B wins 14% with the map only and 45% with full context.

Table 4: Compass results over 100 games per player and input. Success rate with its 95% interval in brackets; progress and SPL run from 0 to 1. Baselines ignore the input, so they have map only.

Player	Success rate		Progress / SPL	
	Map only	Full context	Map only	Full context
Solver	100% [100, 100]	—	1.000 / 1.000	—
Greedy (walls)	37% [28, 46]	—	0.475 / 0.370	—
Greedy	28% [20, 37]	—	0.356 / 0.280	—
Random	4% [1, 8]	—	0.249 / 0.033	—
Jev	32% [23, 41]	57% [47, 66]	0.425 / 0.319	0.683 / 0.541
Laya	2% [0, 5]	12% [6, 19]	0.105 / 0.020	0.264 / 0.105
GLiClass	3% [0, 7]	13% [7, 20]	0.104 / 0.030	0.216 / 0.130
Gemma 4 26B	35% [26, 44]	61% [51, 70]	0.452 / 0.322	0.714 / 0.568
Qwen3.5-4B	14% [8, 21]	45% [35, 55]	0.335 / 0.124	0.661 / 0.387
DeepSeek V4.1 Flash	39% [30, 49]	60% [50, 69]	0.545 / 0.347	0.715 / 0.542
DeepSeek V4.1 Flash (reasoning)	67% [57, 76]	80% [72, 87]	0.787 / 0.618	0.864 / 0.741

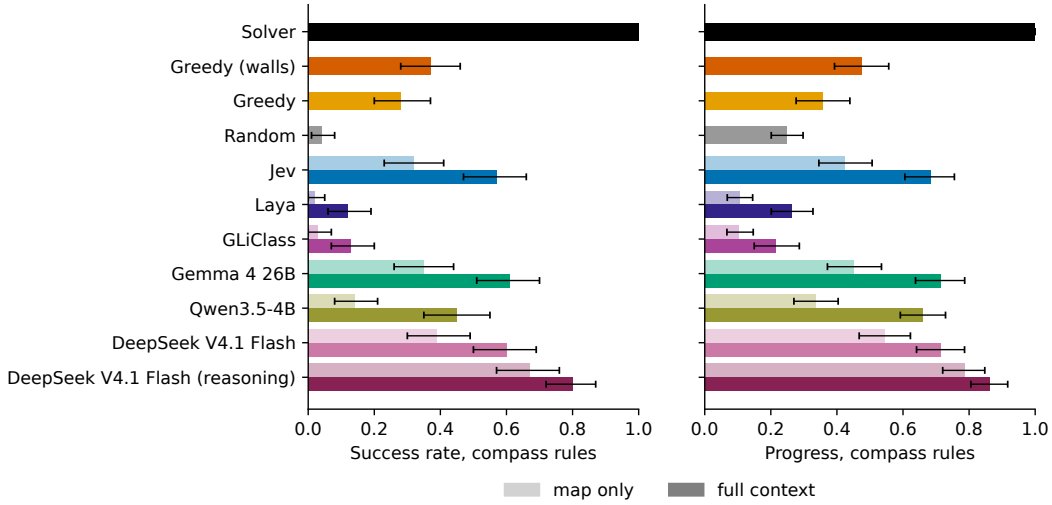


Figure 2: Success rate and progress under compass rules, with the map only (pale bars) and full context (dark bars). Baselines ignore the input and have a single bar. Lines show 95% intervals.

5.2 What each component adds

Figure 3 shows how each model’s success rate changes when a component is included. Added to map only, move outcomes help most models: Jev gains 16 percentage points [9, 23], and the gain is significant for Jev, Laya, GLiClass, Gemma 4 26B and Qwen3.5-4B. For Jev, surroundings (+13 points) and move history (+14) also help significantly. The subgoal helps least (+5 for Jev, not significant).

Removing a single component from full context rarely makes a clear difference. For Jev none of the four removals is significant, and the largest change in progress is 0.06 [0.01, 0.10], for move history. Across all models, only one removal is significant: without move outcomes, Gemma 4 26B wins 10 points fewer [4, 17]. A few other changes, up to 11 points, have intervals that exclude zero but are not significant once corrected for the number of tests. The components seem to overlap, so that the others can make up for a missing one. Appendix D reports all component effects.

5.3 Exam versus games

With full context, Jev answers 90% [87, 93] of the exam positions optimally (Table 5). This matches Gemma 4 26B’s rounded score (paired difference 0 points [−1, 2]) and is 4 points [1, 6] above DeepSeek V4.1 Flash. Only DeepSeek V4.1 Flash (reasoning) scores higher, at 94% [91, 96]. Full context raises every model’s exam score, from 5 points for DeepSeek V4.1 Flash (reasoning) to 30 points for Qwen3.5-4B; every one of these gains has an interval above zero.

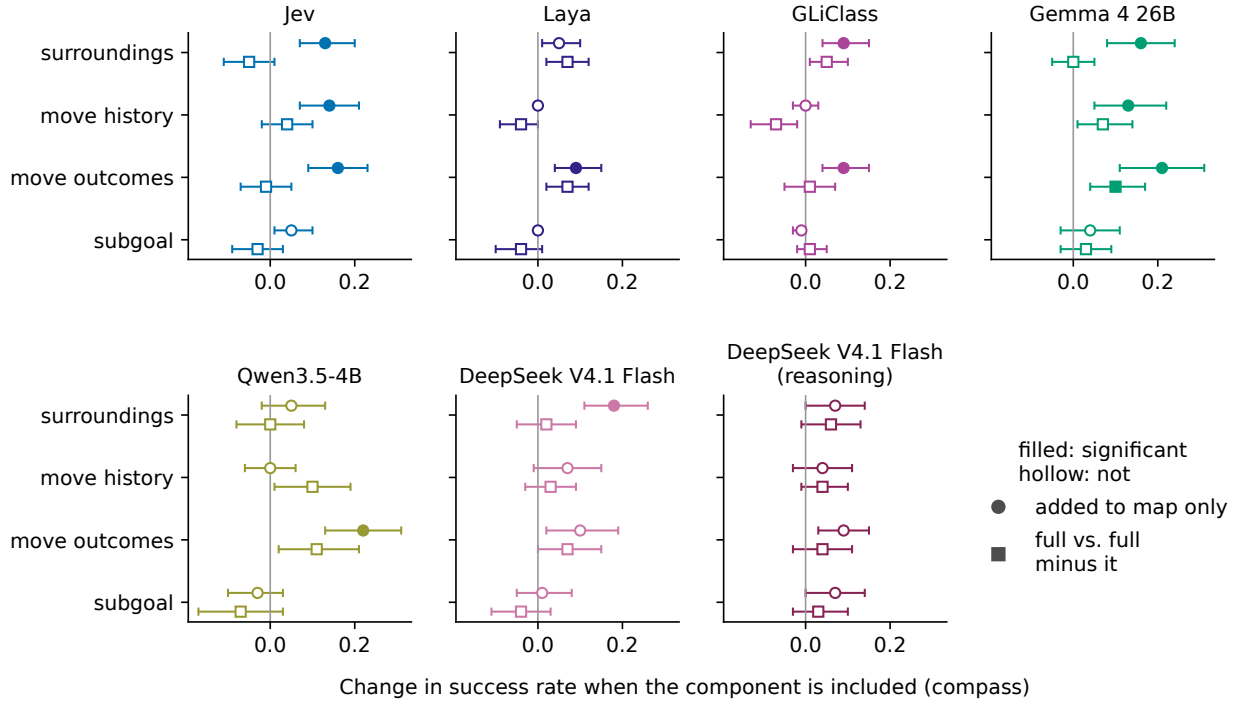


Figure 3: Change in success rate when a component is included (compass rules), with 95% intervals. Circles: map + component compared with map only. Squares: full context compared with full context minus the component. Right of the vertical line, the component helps. Filled markers are significant (McNemar’s test, Holm-corrected).

A high exam score does not guarantee a high success rate. Jev answers 90% of the exam positions optimally but wins only 57% of its games, and in those games 79% of its moves are optimal on average. If every move is counted equally, instead of averaging per game, the share drops to 64%, because lost games are long and full of mistakes. A baseline shows the gap most clearly: Greedy (walls), which never plans, answers 89% of the exam positions optimally but wins only 37% of its games. In most positions, heading straight for the target is optimal. The few positions that need a detour probably decide many games.

Table 5: Exam and game results under compass rules. *Exam*: share of the 495 fixed positions answered with an optimal move, each puzzle counting equally (95% interval in brackets). *Own games: optimal moves*: share of optimal moves in the player’s 100 games with the same input, averaged per game. *Own games: won*: success rate in those games. Baselines ignore the input, so they have map only.

Player	Input	Exam: optimal answers	Own games: optimal moves	Own games: won
Solver	map only	100% [100, 100]	100%	100%
Greedy (walls)	map only	89% [85, 92]	70%	37%
Greedy	map only	73% [68, 77]	32%	28%
Random	map only	30% [25, 35]	30%	4%
Jev	map only	74% [69, 78]	38%	32%
Jev	full context	90% [87, 93]	79%	57%
Laya	map only	27% [22, 32]	15%	2%
Laya	full context	48% [43, 53]	32%	12%
GLiClass	map only	30% [25, 34]	8%	3%
GLiClass	full context	51% [46, 56]	28%	13%
Gemma 4 26B	map only	69% [65, 74]	40%	35%
Gemma 4 26B	full context	90% [87, 93]	79%	61%
Qwen3.5-4B	map only	45% [40, 50]	36%	14%
Qwen3.5-4B	full context	75% [71, 79]	70%	45%
DeepSeek V4.1 Flash	map only	67% [62, 72]	47%	39%
DeepSeek V4.1 Flash	full context	86% [83, 90]	76%	60%
DeepSeek V4.1 Flash (reasoning)	map only	88% [84, 92]	79%	67%
DeepSeek V4.1 Flash (reasoning)	full context	94% [91, 96]	87%	80%

5.4 Blocked moves

Under compass rules with the map only, 82% of Jev’s moves are blocked. Without the move history, a blocked move leaves the request unchanged apart from the order of the options, so Jev can choose the same move into the same wall again, and often does. With full context only 0.4% of its moves are blocked, yet it still loses 43% of its games: avoiding walls is only part of finding the way. Some games get close to the goal and then wander off. With full context, Qwen3.5-4B’s progress is 0.66, but its final-state progress is 0.56.

5.5 Success by level

Success falls as the level rises (Figure 4). At levels 12, 15 and 20, which have ten puzzles each, Jev wins 20%, 10% and 0% of games with full context. DeepSeek V4.1 Flash (reasoning) wins 70%, 40% and 30% with full context, and 50%, 20% and 20% with the map only. Long routes stay hard even with reasoning.

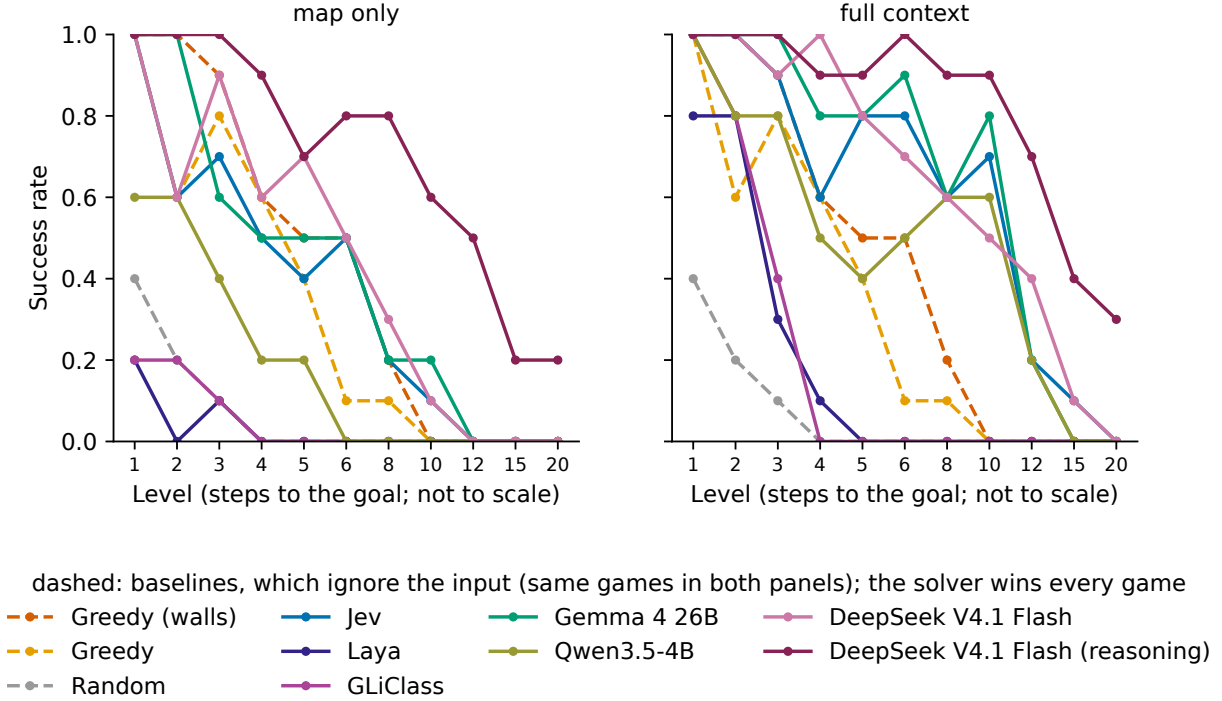


Figure 4: Success rate by level under compass rules. The level is the length of the shortest route in steps; levels are evenly spaced on the axis whatever their value. Levels 1 and 2 have five puzzles each, the others ten.

5.6 Move rules

Longer moves have mixed effects (Figure 5). For Jev with the map only, success rises from 32% under *compass* to 40% under *two-moves* and 47% under *three-moves*, but falls to 26% under *up-to-three-moves*, the rules with the most options (84). These differences between rules were not tested puzzle by puzzle, and for most of them the 95% intervals overlap. With full context, Jev and Gemma 4 26B stay within 8 points of their compass results under every rule. Qwen3.5-4B changes most: with full context its success rate rises from 45% under *compass* to between 58% and 71% under the four sequence rules, but with the map only it falls from 14% to between 3% and 11%. Its gain depends on the move outcomes: under the two three-step rules, full context without them wins only 16% and 18%. Laya and GLiClass stay below 15% under every rule. Because the number of options, the length of their texts and the steps before the model sees the board again all change together, we cannot say which of them causes these effects.

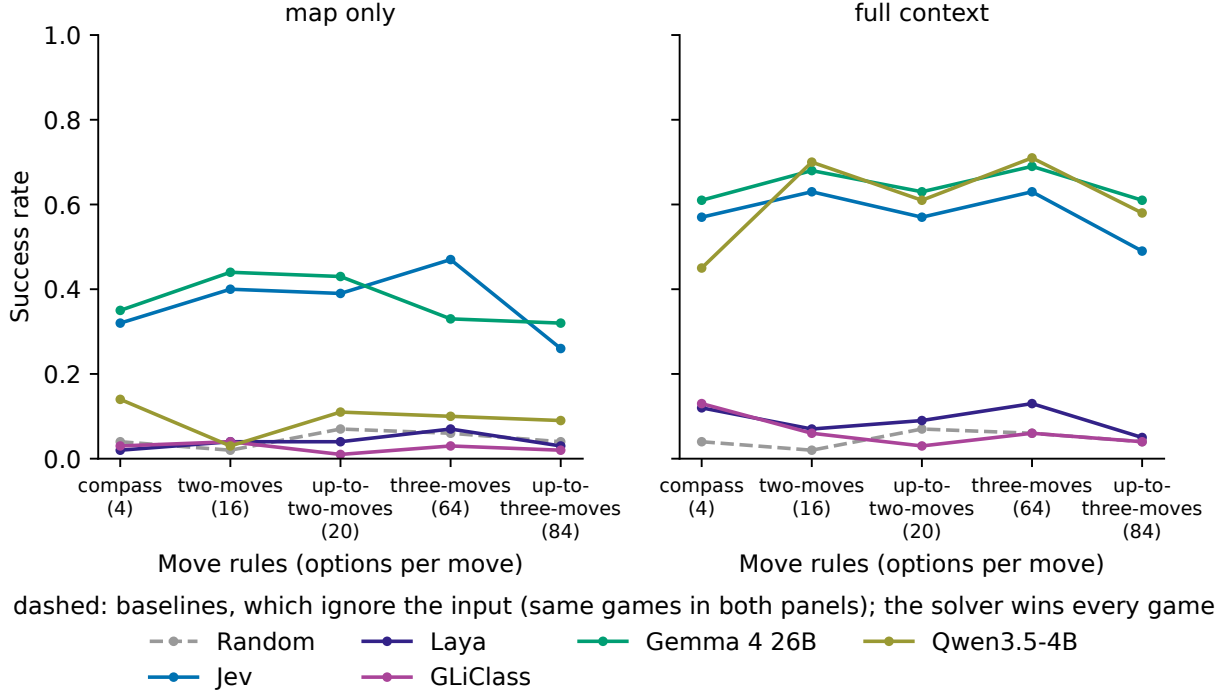


Figure 5: Success rate under the five move rules (options per move in brackets). Both DeepSeek V4.1 Flash setups played compass only and are not shown. Random ignores the input, so its line is the same in both panels; Solver wins every game and is not drawn.

5.7 Probabilities

In its own games, Jev gives its chosen moves higher probabilities than how often they are optimal (Figure 6). Over all 14,642 of its compass moves, under all ten inputs, it gives the chosen move an average probability of 0.72, but only 51% of those moves are optimal; its ECE is 0.21. Its Brier score, 0.283, is worse than the 0.250 it would get by always giving the chosen move a probability of 0.51, its share of optimal moves in the same data. Its probabilities still carry some signal: moves chosen with a probability of 0.9 or more are optimal 74% of the time, against 42% for the rest. On the exam, the same measure gives a much lower ECE of 0.06 with either input, so the gap shows up mainly in the positions its own games lead to.

In their own games, Laya and Qwen3.5-4B have a lower ECE (0.13 and 0.06) and GLiClass a higher one (0.33), although Laya’s Brier score (0.166) is still worse than a constant guess (0.144). A low ECE does not mean good play: only 17.5% of Laya’s chosen moves are optimal, and its low probabilities say so. Jev also returns a confidence score. On every move it equals, to within 0.023, the top probability rescaled so that 0 means all options are equally likely and 1 means certainty, $(p_{\max} - 1/n)/(1 - 1/n)$ for n options [21]. So it adds almost nothing to the probabilities; we cannot explain the small differences.

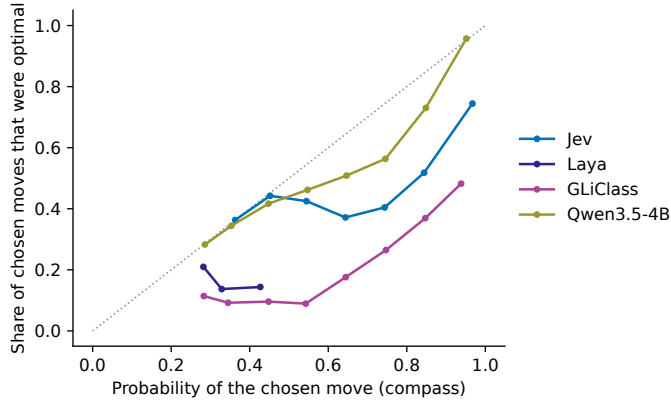


Figure 6: Calibration over all compass games. Moves are grouped by the probability of the chosen move; each point compares a group’s average probability with the share of its moves that were optimal. Points on the dotted diagonal are perfectly calibrated, and points below it are overconfident. Only groups of at least 30 moves are shown. Gemma 4 26B and the DeepSeek V4.1 Flash setups give no probabilities.

5.8 Cost and speed

Median time per answer, counting only the successful call, is 0.50 seconds for Jev, 0.65 for Gemma 4 26B, 0.81 for DeepSeek V4.1 Flash and 6.8 for DeepSeek V4.1 Flash (reasoning). Qwen3.5-4B takes 0.23 seconds on a laptop GPU (Appendix C). Laya and GLiClass, whose compass games ran on a CPU, take about 1.8 seconds. The recorded cost per 100 compass games is \$0.059 for Gemma 4 26B, \$0.047 for DeepSeek V4.1 Flash and \$0.348 for DeepSeek V4.1 Flash (reasoning). We have no dollar costs for Jev or the local models. Services and hardware differ, so these numbers are only a rough guide (Appendix C).

6 Discussion and limitations

Single decisions versus whole games. Several things probably explain why Jev’s high exam score comes with a modest success rate. In its own games, a model reaches positions that a good player would never see, so its mistakes compound [22]. A long game gives many chances to fail: if each of ten moves were right 90% of the time, independently, all ten would be right only 35% of the time. Real games are less simple, because a mistake can be corrected, but the arithmetic shows how a high score per move can sit next to a modest success rate. Finally, as Greedy (walls) shows, most positions are easy, and games are decided by the few that need a detour. We did not measure how much each of these explanations contributes.

What helps. For Jev, full context almost removes blocked moves (0.4%), yet it still loses 43% of its games, so avoiding walls is only part of finding the way. Move outcomes give Jev its largest gain when added to map only (+16 points), though for DeepSeek V4.1 Flash the surroundings help more. Once the other components are present, removing any one of them rarely makes a clear difference; the only significant exception is move outcomes for Gemma 4 26B. We did not separate a component’s content from the extra text it adds.

Limitations.

- We test one small symbolic world, with five or ten puzzles per level and no held-out set. The prompts were written, using a few test games, before the puzzles were generated.
- Each result comes from one game per puzzle. Jev and DeepSeek V4.1 Flash give different answers to identical requests, and Gemma 4 26B and DeepSeek V4.1 Flash (reasoning) probably do too, so a rerun would play different games. The intervals cover the choice of puzzles, not this variation (Appendix C).
- The move rules change several things together (options, text length, steps before the model sees the board again). Two smaller differences also come with them. The subgoal question asks for the “shortest path” under the fixed-length rules and for the way “that takes the fewest turns” under the up-to rules (a turn here is a move); both are scored by moves. And since

the move limit counts moves, longer moves allow more steps: a four-step puzzle allows 8 steps under `compass` but 12 under `three-moves`.

- Levels differ in layout as well as route length, so the level results do not isolate distance.
- Providers, inference limits and hardware differ between models. Laya was used outside the tasks it was trained for. Costs are missing for Jev and the local models.

Code and data. The environment, solver, frozen requests, game records, analysis and leaderboard are at github.com/Exemocar/JevStuff. The leaderboard is also online at exemocar.github.io/JevStuff. The repository explains how to submit a new player and how submissions are checked by replaying their games.

7 Conclusion

System-One Control Bench tests models on single decisions and on whole games, in a world where every best move is known. With full context, the non-generative decision model Jev plays about as well as two chat models, Gemma 4 26B and DeepSeek V4.1 Flash without reasoning, and full context raises its success rate from 32% to 57%. Laya and GLiClass play near chance level with the map only and remain weak with full context. DeepSeek V4.1 Flash (reasoning) plays best, but each answer takes about 7 seconds. For every model, answering single positions well does not guarantee reaching the goal: Jev picks an optimal move in 90% of the exam positions but wins 57% of its games. These results hold for these puzzles, inputs and settings.

AI use

This project was carried out with substantial help from AI assistants, chiefly Anthropic’s Claude models (through Claude Code) and OpenAI’s GPT models (through Codex). Under the author’s direction they wrote most of the code and the analysis scripts, drafted and revised the text of this report, and reviewed one another’s work. The author set the research question, made the design decisions, checked the results and reviewed the manuscript, and takes full responsibility for its contents.

References

- [1] M. Saplin, “TypeSafe Jev Played Chess — And Landed Next to Reasoning Models.” [Online]. Available: <https://dev.to/maximsaplin/typesafe-jev-played-chess-and-landed-next-to-reasoning-models-28ga>
- [2] droidrun, “mobile-jev: a standalone mobile agent for Android using Jev.” [Online]. Available: <https://github.com/droidrun/mobile-jev>
- [3] browser-use, “jev-ultrafast: a browser agent with an indexed action space.” [Online]. Available: <https://github.com/browser-use/jev-ultrafast>
- [4] P. Chopra, “Jev vs Laya vs our local Qwen3 decision model.” [Online]. Available: <https://gist.github.com/paraschopra/48c69b7edb99cd15137524379ccea3a>
- [5] Convai Innovations, “Laya Typed-Decisions: Model Card.” [Online]. Available: <https://huggingface.co/convaiinnovations/laya-typed-decisions>
- [6] I. Stepanov, M. Shtopko, D. Vodanytskyi, O. Lukashov, A. Yavorskyi, and M. Yaroshenko, “GLiClass: Generalist Lightweight Model for Sequence Classification Tasks,” *arXiv preprint arXiv:2508.07662*, 2025.
- [7] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, and S. Kambhampati, “PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change,” in *NeurIPS Datasets and Benchmarks Track*, 2023.
- [8] X. Liu *et al.*, “AgentBench: Evaluating LLMs as Agents,” in *ICLR*, 2024.
- [9] D. Paglieri *et al.*, “BALROG: Benchmarking Agentic LLM and VLM Reasoning On Games,” in *ICLR*, 2025.
- [10] M. Chevalier-Boisvert *et al.*, “BabyAI: A Platform to Study the Sample Efficiency of Grounded Language Learning,” in *International Conference on Learning Representations*, 2019.
- [11] M. Chevalier-Boisvert *et al.*, “Minigrid & Miniworld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks,” *arXiv preprint arXiv:2306.13831*, 2023.
- [12] M. Ahn *et al.*, “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [13] R. Hazra, P. Zuidberg Dos Martires, and L. De Raedt, “SayCanPay: Heuristic Planning with Large Language Models using Learnable Domain Knowledge,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 18, pp. 20123–20133, 2024, doi: [10.1609/aaai.v38i18.29991](https://doi.org/10.1609/aaai.v38i18.29991).
- [14] C. Zheng, H. Zhou, F. Meng, J. Zhou, and M. Huang, “Large Language Models Are Not Robust Multiple Choice Selectors,” in *ICLR*, 2024.
- [15] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” in *ICML*, in PMLR, vol. 70. 2017, pp. 1321–1330.
- [16] Qwen Team, “Qwen3.5: Towards Native Multimodal Agents,” *Qwen blog*, Feb. 2026, [Online]. Available: <https://qwen.ai/blog?id=qwen3.5>
- [17] Gemma Team, “Gemma 4 Technical Report,” *arXiv preprint arXiv:2607.02770*, 2026.
- [18] DeepSeek-AI, “DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression.” [Online]. Available: <https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash>
- [19] OpenRouter, “DeepSeek V4.1 Flash.” [Online]. Available: <https://openrouter.ai/deepseek/deepseek-v4.1-flash>
- [20] P. Anderson *et al.*, “On Evaluation of Embodied Navigation Agents,” *arXiv preprint arXiv:1807.06757*, 2018.
- [21] TypeSafe AI, “Confidence.” [Online]. Available: <https://docs.typesafe.ai/confidence>
- [22] S. Ross, G. Gordon, and D. Bagnell, “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning,” in *International Conference on Artificial Intelligence and Statistics*, in PMLR, vol. 15. 2011, pp. 627–635.

A Two example requests

Both requests below are for the position in Figure 1, shown in full; lines wrap only to fit the page. Every model gets the same three parts: the state, the question and the options. Jev receives them as the fields of its API. Chat models receive them in one message, after this instruction: “You are playing a puzzle on a grid. You are given the state of the game, a question and a list of options, each with an id such as option_3. Reply with the id of the one option you choose.” The repository’s `examples/` directory holds the request for every input condition and move rule.

A.1 Compass rules, map only

STATE

You move one cell per turn: north (up), south (down), east (right) or west (left). Walls (#) block you, and walking into one wastes the move. Stepping onto the key (K) picks it up. A locked door (D) opens when you walk into it carrying the key, and you step through; without the key it blocks you like a wall. Reaching the goal (G) wins.

Map, with column numbers along the top and row numbers down the side (# = wall, . = floor, A = you, G = goal, K = key, D = locked door). A position (x, y) is column x, row y:

```
0123456789
0 #####
1 #.....A.#
2 #...#.K.#
3 #D...#..#
4 #G#.#...##
5 #####
```

You are A at (7, 1). You are carrying nothing.

Objects: goal G at (1, 4); key K at (7, 2); locked door D at (1, 3).

QUESTION

What is the best next move?

OPTIONS

option_1: move south (down)
option_2: move north (up)
option_3: move west (left)
option_4: move east (right)

A.2 Up-to-two-moves rules, full context

With full context the state adds the surroundings and the move history, each option says what it would do, and the question names the next target.

STATE

Each turn you choose a path of one or two steps, taken one after another. Each step goes one cell: north (up), south (down), east (right) or west (left). Walls (#) block you, and walking into one wastes that step. Stepping onto the key (K) picks it up. A locked door (D) opens when you walk into it carrying the key, and you step through; without the key it blocks you like a wall. Reaching the goal (G) wins at once, and any steps left are not taken.

Map, with column numbers along the top and row numbers down the side (# = wall, . = floor, A = you, G = goal, K = key, D = locked door). A position (x, y) is column x, row y:

```
0123456789
0 #####
1 #.....A.#
2 #...#.K.#
3 #D...#..#
4 #G#.#...##
5 #####
```

You are A at (7, 1). You are carrying nothing.

Objects: goal G at (1, 4); key K at (7, 2); locked door D at (1, 3).

North of you is a wall. South of you is the key. East of you is open floor. West of you is open floor. The goal is 6 west and 3 south of you. The key is 1 south of you. The locked door is 6 west and 2 south of you.

Moves so far:

1. east: you end at (7, 1)
2. north,north: you end where you started, at (7, 1)

QUESTION

Your next target is the key K at (7, 2). Which move starts the way to the goal through it that takes the fewest turns?

OPTIONS

- option_1: move north (up), then south (down): you end at (7, 2) and pick up the key
- option_2: move south (down), then south (down): you end at (7, 3) and pick up the key
- option_3: move west (left), then east (right): you end where you started, at (7, 1)
- option_4: move east (right), then south (down): you end at (8, 2)
- option_5: move south (down), then east (right): you end at (8, 2) and pick up the key
- option_6: move east (right), then east (right): you end at (8, 1)
- option_7: move north (up), then west (left): you end at (6, 1)
- option_8: move west (left): you end at (6, 1)
- option_9: move south (down), then west (left): you end at (6, 2) and pick up the key
- option_10: move east (right): you end at (8, 1)
- option_11: move west (left), then west (left): you end at (5, 1)
- option_12: move north (up), then east (right): you end at (8, 1)
- option_13: move west (left), then south (down): you end at (6, 2)
- option_14: move east (right), then west (left): you end where you started, at (7, 1)
- option_15: move east (right), then north (up): you end at (8, 1)
- option_16: move north (up): you end where you started, at (7, 1)
- option_17: move south (down): you end at (7, 2) and pick up the key
- option_18: move north (up), then north (up): you end where you started, at (7, 1)
- option_19: move west (left), then north (up): you end at (6, 1)
- option_20: move south (down), then north (up): you end where you started, at (7, 1) and pick up the key

B Hypotheses written before the main runs

Table 6: The seven hypotheses about Jev written before the main runs, checked as written. They keep their original wording, which uses older names: “won rate” is the success rate, “interaction history” the move history, “simulated action outcomes” the move outcomes, “explicit subgoal” the subgoal, and p(chosen) the probability of the chosen move; map+X is the map plus one component, and everything-X is full context minus one.

Hypothesis (original wording)	Result	Met
H1: under map only, Jev beats random but not greedy (walls), on won rate and progress	success rate vs Random: +0.28 [+0.19, +0.37]; progress vs Random: +0.18 [+0.09, +0.26]; success rate vs Greedy (walls): -0.05 [-0.12, +0.02]; progress vs Greedy (walls): -0.05 [-0.11, +0.01]; the comparison with Greedy (walls) is inconclusive	partly
H2: Jev's wins fall with the level; no condition wins half of the planning puzzles	success falls by 0.046 to 0.061 per route step (10 inputs); planning puzzles are the 63 that Greedy (walls) loses; the best input on them wins 0.40 (full context minus surroundings)	yes
H3: simulated action outcomes help most (best map+X, worst everything-X, on progress)	progress, map + one component: move outcomes 0.61, move history 0.58, surroundings 0.57, subgoal 0.46; lowest full context minus one: full context minus move history 0.63	partly
H4: explicit subgoal is second among additions, and helps mostly from level 3 up	ranks 4 of 4; progress gain +0.02 at levels 3 and up, +0.10 at levels 1-2	no
H5: interaction history alone adds almost nothing (within 0.05 progress of map)	progress change +0.15 [+0.11, +0.21]	no
H6: full context has the best progress of the ten conditions and wins at least 28/100	best progress: full context minus subgoal (0.73); full context wins 57/100	partly
H7: Jev is overconfident, but its p(chosen) still ranks its moves	chosen move's mean probability 0.72, but 0.51 of chosen moves optimal; optimal at probability 0.9 or more: 0.74 (3895 moves), below 0.9: 0.42; yet a higher-probability bin is not always more often optimal	partly

C Setup and reproducibility

Table 7: Model IDs and settings. Model versions on the laptop: Laya 55cf4c4, GLiClass 668dd0d, Qwen3.5-4B 851bf6e. Software versions are locked in the repository’s `uv.lock`.

Model	Model ID / where it ran	Precision and limits
Jev	jev-1.13.0 / TypeSafe	Precision not reported.
Laya	convainnovations/laya, subfolder typed-decisions (also published as convainnovations/laya-typed-decisions [5]) / local	Precision not recorded; reads up to 8,192 tokens, 48 per option.
GLiClass	knowledgator/gliclass-modern-large-v3.0 / local	Precision not recorded; reads up to 8,192 tokens and 128 labels.
Qwen3.5-4B	Qwen/Qwen3.5-4B / local	bfloat16; reasoning off; writes nothing.
Gemma 4 26B	google/gemma-4-26b-a4b-it / OpenRouter, DeepInfra	Precision not reported; reasoning off; answer up to 64 tokens.
DeepSeek V4.1 Flash	deepseek/deepseek-v4.1-flash / OpenRouter, DeepInfra	Precision not reported; answer up to 64 tokens, plus up to 1,024 reasoning tokens with reasoning.

Hardware. The Laya and GLiClass compass games (25 September 2026) ran on the CPU of a second computer, without a GPU; its exact model was not recorded. Every other local run (the Laya and GLiClass sequence-rule games from 26 September on, all Qwen3.5-4B games, and all exam runs) used an Acer Predator PHN16S-71 laptop: Intel Core Ultra 9 275HX (24 cores), 32 GB of memory, NVIDIA RTX 5070 Ti Laptop GPU (12 GB), Windows 11, Python 3.11.9, CUDA 13.0, torch 2.14.0, transformers 5.17.0 and gliclass 0.1.20. The runs took place between 23 September and 6 October 2026; the result file names in `analysis/manifest.toml` give the dates.

Calls. Chat requests set neither temperature nor `top_p`, so the provider’s defaults apply. DeepSeek V4.1 Flash with reasoning is called `deepseek-v4.1-flash-think` in the repository. A refused OpenRouter call is retried up to four times, after 5, 15, 30 and 60 seconds; a call times out after 120 seconds without reasoning and 600 with it. A refused Jev call is retried twice, after one and two seconds; a Jev call times out after 120 seconds and is not retried.

Time and cost. Times count only the successful call, without retries, waiting or model loading. Costs add up the provider-reported cost of each successful call; failed earlier attempts have no recorded cost. Different hosts and devices mean the times are not a controlled comparison.

Local models. Laya gets the full state as the first argument of its `system_one` call, and the program raises Laya’s overall token limits as far as each request needs. An option longer than Laya’s fixed 48 tokens would be refused rather than cut; none was. GLiClass gives each option an independent score between 0 and 1, which we scale to sum to 1. Qwen3.5-4B reads the chat with reasoning off, followed by the start of the answer, and we read the probability of each option number from it.

Exam. The exam positions were drawn once, with seed 0: 100 start, 95 on-route, 144 off-route, 88 after-blocked and 68 late. Where a kind of position cannot be made for a puzzle (for example, an after-blocked position in a puzzle whose route never passes a wall), an extra off-route or after-blocked position takes its place, sometimes on a board already used. Random always picks the option in one fixed place in the shuffled list, which is itself a random pick. Choosing uniformly at random would be optimal on 27% of these positions; Random scored 30%.

One question changed. On 26 September the subgoal question for the up-to rules was reworded to ask for the way that takes the fewest turns (moves). Every game that used the old wording was replayed by every player, and only the replays are used here.

Replay checks. Rebuilding every request from the recorded games, both with the code of the day and with the released code, gives byte-identical states, questions and options, apart from that one reworded question. Asking GLiClass, Laya and Qwen3.5-4B again at 529 recorded positions (176, 124 and 229) gave exactly the same probabilities, including with Laya’s newer version 7b928d8. Qwen3.5-4B’s games under the four sequence rules were also checked this way, at 169 (two-moves), 169 (up-to-two-moves), 148 (three-moves) and 151 (up-to-three-moves) positions, with the same result. Jev, asked three times at ten positions, gave probabilities that differed by up to 0.14, and replayed at 92 positions it chose the same move at 91. DeepSeek V4.1 Flash gave up to four different answers in five identical calls. So a rerun of Jev or DeepSeek V4.1 Flash, and probably of Gemma 4 26B and DeepSeek V4.1 Flash (reasoning), would play different games, and the results here are one game per puzzle.

This report describes release v1.0.0 of the repository.

D All component effects

Table 8: Effect of including each component, for every model (compass rules, 100 paired puzzles). *Added to map only* compares map + component with map only; the second group compares full context with full context minus the component. Positive values mean the component helps. Success-rate changes are in percentage points, progress changes on the 0–1 scale; brackets give 95% intervals. p is McNemar’s exact test on games won, Holm-corrected over each model’s eight tests.

Component	Added to map only			Full context vs. full context minus it		
	Success (points)	Progress	p	Success (points)	Progress	p
Jev						
surroundings	+13 [7, 20]	+0.14 [0.09, 0.20]	0.001	−5 [−11, 1]	−0.02 [−0.06, 0.02]	0.719
move history	+14 [7, 21]	+0.15 [0.11, 0.21]	<0.001	+4 [−2, 10]	+0.06 [0.01, 0.10]	1.000
move outcomes	+16 [9, 23]	+0.18 [0.13, 0.24]	<0.001	−1 [−7, 5]	0.00 [−0.04, 0.05]	1.000
subgoal	+5 [1, 10]	+0.03 [0.00, 0.07]	0.312	−3 [−9, 3]	−0.04 [−0.09, 0.00]	1.000
Laya						
surroundings	+5 [1, 10]	+0.10 [0.06, 0.16]	0.312	+7 [2, 12]	+0.12 [0.06, 0.17]	0.109
move history	0 [0, 0]	−0.02 [−0.05, 0.00]	1.000	−4 [−9, 0]	+0.02 [−0.03, 0.07]	0.875
move outcomes	+9 [4, 15]	+0.08 [0.03, 0.14]	0.031	+7 [2, 12]	+0.09 [0.03, 0.14]	0.109
subgoal	0 [0, 0]	−0.01 [−0.04, 0.01]	1.000	−4 [−10, 1]	−0.02 [−0.07, 0.02]	0.875
GLiClass						
surroundings	+9 [4, 15]	+0.11 [0.05, 0.17]	0.031	+5 [1, 10]	+0.08 [0.04, 0.13]	0.312
move history	0 [−3, 3]	+0.04 [0.01, 0.08]	1.000	−7 [−13, −2]	−0.09 [−0.13, −0.05]	0.234
move outcomes	+9 [4, 15]	+0.12 [0.06, 0.18]	0.031	+1 [−5, 7]	−0.05 [−0.12, 0.02]	1.000
subgoal	−1 [−3, 0]	+0.05 [0.01, 0.09]	1.000	+1 [−2, 5]	+0.01 [−0.02, 0.05]	1.000
Gemma 4 26B						
surroundings	+16 [8, 24]	+0.18 [0.11, 0.25]	0.003	0 [−5, 5]	+0.01 [−0.03, 0.06]	1.000
move history	+13 [5, 22]	+0.12 [0.06, 0.19]	0.038	+7 [1, 14]	+0.05 [0.00, 0.10]	0.262
move outcomes	+21 [11, 31]	+0.22 [0.14, 0.30]	<0.001	+10 [4, 17]	+0.09 [0.04, 0.13]	0.038
subgoal	+4 [−3, 11]	+0.03 [−0.03, 0.09]	1.000	+3 [−3, 9]	0.00 [−0.04, 0.04]	1.000
Qwen3.5-4B						
surroundings	+5 [−2, 13]	+0.13 [0.06, 0.20]	1.000	0 [−8, 8]	+0.02 [−0.03, 0.07]	1.000
move history	0 [−6, 6]	+0.05 [−0.01, 0.11]	1.000	+10 [1, 19]	+0.07 [0.01, 0.14]	0.383
move outcomes	+22 [13, 31]	+0.22 [0.15, 0.29]	<0.001	+11 [2, 21]	+0.10 [0.03, 0.17]	0.303
subgoal	−3 [−10, 3]	−0.07 [−0.13, −0.01]	1.000	−7 [−17, 3]	−0.03 [−0.10, 0.05]	1.000
DeepSeek V4.1 Flash						
surroundings	+18 [11, 26]	+0.16 [0.11, 0.23]	<0.001	+2 [−5, 9]	−0.01 [−0.05, 0.04]	1.000
move history	+7 [−1, 15]	+0.09 [0.02, 0.16]	0.717	+3 [−3, 9]	+0.02 [−0.01, 0.06]	1.000
move outcomes	+10 [2, 19]	+0.13 [0.06, 0.19]	0.290	+7 [0, 15]	+0.08 [0.02, 0.14]	0.711
subgoal	+1 [−5, 8]	−0.01 [−0.07, 0.04]	1.000	−4 [−11, 3]	−0.07 [−0.13, −0.01]	1.000
DeepSeek V4.1 Flash (reasoning)						
surroundings	+7 [0, 14]	+0.05 [0.00, 0.10]	0.646	+6 [−1, 13]	+0.03 [−0.02, 0.07]	0.730
move history	+4 [−3, 11]	+0.02 [−0.03, 0.08]	1.000	+4 [−1, 10]	+0.02 [−0.01, 0.06]	1.000
move outcomes	+9 [3, 15]	+0.05 [0.01, 0.09]	0.094	+4 [−3, 11]	+0.02 [−0.03, 0.06]	1.000
subgoal	+7 [0, 14]	+0.04 [−0.01, 0.09]	0.646	+3 [−3, 10]	+0.01 [−0.03, 0.06]	1.000